

Day 8: Functions in Python (Detailed Notes for Data Analysis Students)

What is a Function?

A **function** is a reusable block of code that performs a specific task. Instead of writing the same code multiple times, you can write it once inside a function and call it whenever needed.

REAL-LIFE EXAMPLE

Think of a **calculator**:

- Press **+** → Adds numbers
- Press **-** → Subtracts numbers

Each operation works like a function.

Why Use Functions?

Functions help you:

- Avoid writing duplicate code
 - Improve code readability
 - Make programs easier to maintain
 - Reuse code multiple times
 - Organize large projects
-

Types of Functions

1. Built-in Functions
 2. User-Defined Functions
-

1. Built-in Functions

Python provides many ready-made functions.

Examples:

```
print("Hello")

numbers = [10, 20, 30]
```

```
print(len(numbers))
print(max(numbers))
print(min(numbers))
print(sum(numbers))
```

Output:

```
3
30
10
60
```

2. User-Defined Functions

Functions created by the programmer.

SYNTAX

```
def function_name():
    statements
```

EXAMPLE 1

```
def greet():
    print("Welcome to Python")

greet()
```

Output

```
Welcome to Python
```

Calling a Function Multiple Times

```
def greet():
    print("Hello")

greet()
greet()
greet()
```

Output

```
Hello  
Hello  
Hello
```

Function with Parameters

Parameters receive values.

```
def greet(name):  
  
    print("Welcome", name)  
  
greet("Rahul")  
greet("Amit")
```

Output

```
Welcome Rahul  
Welcome Amit
```

Multiple Parameters

```
def employee(name, salary):  
  
    print(name, salary)  
  
employee("Rahul", 50000)
```

Output

```
Rahul 50000
```

Return Statement

Returns a value from the function.

```
def add(a, b):  
  
    return a + b
```

```
result = add(10, 20)

print(result)
```

Output

```
30
```

Difference Between print() and return

Using `print()`:

```
def add():

    print(10 + 20)

add()
```

Output

```
30
```

Using `return`:

```
def add():

    return 10 + 20

result = add()

print(result)
```

Output

```
30
```

Interview Tip: Use `return` when the result will be used later in your program.

Default Parameters

```
def greet(name="Guest"):

    print("Welcome", name)
```

```
greet()  
greet("Rahul")
```

Output

```
Welcome Guest  
Welcome Rahul
```

Keyword Arguments

```
def employee(name, salary):  
  
    print(name, salary)  
  
employee(salary=50000, name="Rahul")
```

Output

```
Rahul 50000
```

Variable Scope

LOCAL VARIABLE

```
def demo():  
  
    x = 10  
  
    print(x)  
  
demo()
```

`x` can only be used inside the function.

GLOBAL VARIABLE

```
x = 100  
  
def demo():  
  
    print(x)
```

```
demo()
```

Output

```
100
```

Lambda Functions (Introduction)

A lambda function is a small anonymous function.

```
square = lambda x: x * x

print(square(5))
```

Output

```
25
```

Real Data Analyst Examples

EXAMPLE 1: TOTAL SALES

```
def total_sales(sales):

    return sum(sales)

sales = [12000, 25000, 30000]

print(total_sales(sales))
```

Output

```
67000
```

EXAMPLE 2: AVERAGE SALES

```
def average_sales(sales):

    return sum(sales) / len(sales)

sales = [12000, 25000, 30000]
```

```
print(average_sales(sales))
```

Output

```
22333.33
```

EXAMPLE 3: EMPLOYEE BONUS

```
def calculate_bonus(salary):  
  
    return salary * 0.10  
  
print(calculate_bonus(50000))
```

Output

```
5000
```

EXAMPLE 4: GRADE CALCULATOR

```
def grade(marks):  
  
    if marks >= 90:  
        return "A"  
  
    elif marks >= 75:  
        return "B"  
  
    elif marks >= 60:  
        return "C"  
  
    else:  
        return "Fail"  
  
print(grade(82))
```

Output

```
B
```

Mini Project 1: Calculator

```
def add(a, b):  
  
    return a + b  
  
def subtract(a, b):  
  
    return a - b  
  
def multiply(a, b):  
  
    return a * b  
  
def divide(a, b):  
  
    return a / b  
  
print(add(10, 5))  
print(subtract(10, 5))  
print(multiply(10, 5))  
print(divide(10, 5))
```

Mini Project 2: Employee Salary Report

```
def salary_report(name, salary):  
  
    print("Employee :", name)  
    print("Salary   :", salary)  
  
salary_report("Rahul", 50000)
```

Mini Project 3: Profit Calculator

```
def profit(sales, cost):  
  
    return sales - cost  
  
print(profit(100000, 70000))
```


Output

```
30000
```

Mini Project 4: Email Validator

```
def validate_email(email):  
  
    return email.endswith("@gmail.com")  
  
print(validate_email("abc@gmail.com"))
```

Output

```
True
```

Practice Questions

BASIC

1. Create a function to print your name.
 2. Create a function to print "Hello Python".
 3. Call a function three times.
 4. Create a function without parameters.
 5. Create a function with one parameter.
-

PARAMETERS

6. Pass employee name.
 7. Pass salary.
 8. Pass city.
 9. Pass age.
 10. Pass product name.
-

RETURN

11. Return sum.
12. Return average.
13. Return maximum number.
14. Return minimum number.

15. Return profit.

DEFAULT ARGUMENTS

16. Welcome user.

17. Default city.

18. Default salary.

19. Default department.

20. Default bonus.

LAMBDA

21. Square of a number.

22. Cube of a number.

23. Add two numbers.

24. Multiply two numbers.

25. Find larger number.

BUSINESS PROBLEMS

26. Employee bonus calculator.

27. Sales total calculator.

28. Attendance checker.

29. Invoice calculator.

30. Profit & loss calculator.

31. Tax calculator.

32. EMI calculator.

33. Salary increment calculator.

34. Customer discount calculator.

35. Sales target checker.

Assignment (Data Analyst Level)

Create an Employee Analysis System.

Input:

```
employees = [  
    ("Rahul", 50000),  
    ("Amit", 65000),
```

```
        ("Priya", 55000)
    ]
```

Create functions to:

- Print employee details
- Calculate total salary
- Calculate average salary
- Find highest salary
- Find lowest salary

Expected Output:

```
Total Salary : 170000
Average Salary : 56666.67
Highest Salary : 65000
Lowest Salary : 50000
```

Interview Questions

1. What is a function?
2. Why do we use functions?
3. Difference between built-in and user-defined functions?
4. What is a parameter?
5. What is an argument?
6. What is the return statement?
7. Difference between `print()` and `return`?
8. What are default arguments?
9. What are keyword arguments?
10. What is variable scope?
11. What is a local variable?
12. What is a global variable?
13. What is a lambda function?
14. Why are functions important in Data Analysis?
15. Give a real-world example where functions reduce code duplication.

Learning Outcome

After Day 8, you should be able to:

- ✓ Create user-defined functions
- ✓ Use parameters and arguments
- ✓ Return values from functions
- ✓ Work with default and keyword arguments
- ✓ Understand local and global variables
- ✓ Write simple lambda functions
- ✓ Build reusable business logic for Data Analysis and automation